WEEK-9

OSSP - SKILL Content Delivery Concepts
Course Code: 25CS2104E

TASK 1

Objective / Skill:

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Implementation / Work:

```
/*
 * Week 9 - Task 1: Standard Input Redirection (<)
 * Objective: To Parse Input Redirection, Open Files, Redirect stdin,
 *            Handle Missing Files, Restore Streams, Test File Inputs.
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>

void test_input_redirection(const char *filename) {
    printf("[Testing Input Redirection: '< %s']\n", filename);

    int saved_stdin = dup(STDIN_FILENO);
    int fd = open(filename, O_RDONLY);
    if (fd < 0) {
        printf("  [Handled Error] Failed to open '%s': %s (errno %d)\n",
               filename, strerror(errno), errno);
        close(saved_stdin);
        return;
    }

    if (dup2(fd, STDIN_FILENO) < 0) {
        perror("dup2 failed");
        close(fd);
        close(saved_stdin);
        return;
    }
    close(fd);

    char line[128];
    if (fgets(line, sizeof(line), stdin)) {
        printf("  [Read from Redirected Stdin]: %s", line);
    }

    // Restore standard stream
    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);
    clearerr(stdin);
    printf("  [Stream Restored] stdin successfully reconnected to terminal.\n");
}

int main() {
    printf("=== Week 9 Task 1: Input Redirection Parsing & Handling ===\n\n");

    // Create test input file
    FILE *f = fopen("test_input.txt", "w");
    if (f) {
        fputs("OSSP Week 9 Input Redirection Sample Line\n", f);
        fclose(f);
    }

    test_input_redirection("test_input.txt");
    printf("\n");
    test_input_redirection("non_existent_file.txt");

    printf("\nInput redirection parsing, execution, and error recovery verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s9.1.c
charith@DESKTOP-RBQQSS:~$ gcc s9.1.c -o s9.1
charith@DESKTOP-RBQQSS:~$ ./s9.1
=== Week 9 Task 1: Input Redirection Parsing & Handling ===

[Testing Input Redirection: '< test_input.txt']
  [Read from Redirected Stdin]: OSSP Week 9 Input Redirection Sample Line
  [Stream Restored] stdin successfully reconnected to terminal.

[Testing Input Redirection: '< non_existent_file.txt']
  [Handled Error] Failed to open 'non_existent_file.txt': No such file or directory (errno 2)

Input redirection parsing, execution, and error recovery verified!
charith@DESKTOP-RBQQSS:~$
```

TASK 2

Objective / Skill:

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

Implementation / Work:

```c
/*
 * Week 9 - Task 2: Standard Output Redirection (>)
 * Objective: To Parse Output Redirection, Create Output Files, Redirect stdout,
 *            Handle Permissions, Verify Output Data, Test Redirection.
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("=== Week 9 Task 2: Output Redirection Parsing & Verification ===\n\n");
    const char *outfile = "test_output.txt";
    printf("Redirecting STDOUT to '%s' (truncate mode)...\n", outfile);

    int saved_stdout = dup(STDOUT_FILENO);
    int fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open failed");
        return 1;
    }

    fflush(stdout);
    dup2(fd, STDOUT_FILENO);
    close(fd);

    // This output is captured into the file
    printf("OSSP Week 9 Task 2: Output redirection verified successfully.\n");
    printf("Generated by redirected stdout stream.\n");
    fflush(stdout);

    // Restore stdout
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("STDOUT restored to terminal!\n");
    printf("Verifying file content on disk:\n----------------------------------------\n");
    FILE *f = fopen(outfile, "r");
    if (f) {
        int ch;
        while ((ch = fgetc(f)) != EOF) putchar(ch);
        fclose(f);
    }
    printf("----------------------------------------\nOutput redirection verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s9.2.c
charith@DESKTOP-RBQQSS:~$ gcc s9.2.c -o s9.2
charith@DESKTOP-RBQQSS:~$ ./s9.2
=== Week 9 Task 2: Output Redirection Parsing & Verification ===

Redirecting STDOUT to 'test_output.txt' (truncate mode)...
STDOUT restored to terminal!
Verifying file content on disk:
----------------------------------------
OSSP Week 9 Task 2: Output redirection verified successfully.
Generated by redirected stdout stream.
----------------------------------------
Output redirection verified!
charith@DESKTOP-RBQQSS:~$
```

TASK 3

Objective / Skill:

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Implementation / Work:

```c
/*
 * Week 9 - Task 3: Output Append Redirection (>>)
 * Objective: To Parse Append Operator, Open Files in Append Mode, Redirect Streams,
 *            Preserve Existing Data, Validate Appends, Test Scenarios.
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

void append_line(const char *outfile, const char *text) {
    int saved_stdout = dup(STDOUT_FILENO);
    int fd = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0) { perror("open append failed"); return; }
    fflush(stdout);
    dup2(fd, STDOUT_FILENO);
    close(fd);
    printf("%s\n", text);
    fflush(stdout);
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
}

int main() {
    printf("=== Week 9 Task 3: Append Operator (>>) Verification ===\n\n");
    const char *outfile = "append_log.txt";
    unlink(outfile);

    printf("[Run 1] Appending initial record...\n");
    append_line(outfile, "[Record 1] Initializing system log...");

    printf("[Run 2] Appending second record...\n");
    append_line(outfile, "[Record 2] Appending secondary audit trail...");

    printf("\nVerifying that existing data was preserved across appends:\n");
    printf("----------------------------------------\n");
    FILE *f = fopen(outfile, "r");
    if (f) {
        char line[128];
        while (fgets(line, sizeof(line), f)) printf("  %s", line);
        fclose(f);
    }
    printf("----------------------------------------\nAppend operator (>>) validated successfully!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s9.3.c
charith@DESKTOP-RBQQSS:~$ gcc s9.3.c -o s9.3
charith@DESKTOP-RBQQSS:~$ ./s9.3
=== Week 9 Task 3: Append Operator (>>) Verification ===

[Run 1] Appending initial record...
[Run 2] Appending second record...

Verifying that existing data was preserved across appends:
----------------------------------------
  [Record 1] Initializing system log...
  [Record 2] Appending secondary audit trail...
----------------------------------------
Append operator (>>) validated successfully!
charith@DESKTOP-RBQQSS:~$
```

TASK 4

Objective / Skill:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Implementation / Work:

```c
/*
 * Week 9 - Task 4: Stderr Redirection (2>)
 * Objective: To Parse stderr Redirection, Redirect Error Streams, Capture Error Output,
 *            Verify File Contents, Handle Failures, Test Error Cases.
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    printf("=== Week 9 Task 4: Stderr Redirection (2>) ===\n\n");
    const char *errfile = "error_capture.log";

    printf("Redirecting STDERR to '%s'...\n", errfile);
    int saved_stderr = dup(STDERR_FILENO);
    int fd = open(errfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open failed"); return 1; }

    dup2(fd, STDERR_FILENO);
    close(fd);

    // This error output is captured in the error log file
    fprintf(stderr, "[ERROR ALERT] Simulation of critical system failure: Invalid configuration code #503\n");
    fflush(stderr);

    // Restore stderr
    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stderr);

    printf("STDERR restored to terminal!\n");
    printf("Verifying captured error log:\n----------------------------------------\n");
    FILE *f = fopen(errfile, "r");
    if (f) {
        char line[128];
        while (fgets(line, sizeof(line), f)) printf("  %s", line);
        fclose(f);
    }
    printf("----------------------------------------\nStderr redirection verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s9.4.c
charith@DESKTOP-RBQQSS:~$ gcc s9.4.c -o s9.4
charith@DESKTOP-RBQQSS:~$ ./s9.4
=== Week 9 Task 4: Stderr Redirection (2>) ===

Redirecting STDERR to 'error_capture.log'...
STDERR restored to terminal!
Verifying captured error log:
----------------------------------------
  [ERROR ALERT] Simulation of critical system failure: Invalid configuration code #503
----------------------------------------
Stderr redirection verified!
charith@DESKTOP-RBQQSS:~$
```